

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..25.3.5

CUSTOM PROVIDERS

 Copy as Markdown

Custom providers

INFO

This page documents the legacy `serverpod_auth` module. To move an existing app to the current authentication framework, see [Migrate from legacy auth](#).

The legacy `serverpod_auth` module makes it easy to implement custom authentication providers. This allows you to leverage all the existing providers supplied by the module along with the specific providers your project requires.

Server setup

After successfully authenticating a user through a custom provider, an auth token can be created and connected to the user to preserve the authenticated user's permissions. This token is used to identify the user and facilitate endpoint authorization validation. The token can be removed when the user signs out to prevent further access.

Connect user

The authentication module provides methods to find or create users. This ensures that all authentication tokens from the same user are connected.

Users can be identified either by their email through the `Users.findUserByEmail(...)` method or by a unique identifier through the `Users.findUserByIdentifier(...)` method.

If no user is found, a new user can be created through the `Users.createUser(...)` method.

```
UserInfo? userInfo;  
userInfo = await Users.findUserByEmail(session, email);  
userInfo ??= await Users.findUserByIdentifier(session, userIdentifier);  
if (userInfo == null) {  
    userInfo = UserInfo(  

```



```

        userIdentifier: userIdentifier,
        userName: name,
        email: email,
        blocked: false,
        created: DateTime.now().toUtc(),
        scopeNames: [],
    );
    userInfo = await Users.createUser(session, userInfo, _authMethod);
}

```

The example above tries to find a user by email and user identifier. If no user is found, a new user is created with the provided information.

NOTE

For many authentication platforms the `userIdentifier` is the user's email, but it can also be another unique identifier such as a phone number or a social security number.

Custom identification methods

If other identification methods are required you can easily implement them by accessing the database directly. The `UserInfo` model can be interacted with in the same way as any other model with a database in Serverpod.

```

var userInfo = await UserInfo.db.findFirstRow(
    session,
    where: (t) => t.fullName.equals(name),
);

```

The example above shows how to find a user by name using the `UserInfo` model.

Create auth token

When a user has been found or created, an auth token that is connected to the user should be created.

To create an auth token, call the `signInUser` method in the `UserAuthentication` class, accessible as a static method, e.g. `UserAuthentication.signInUser`.

The `signInUser` method takes four arguments: the first is the session object, the second is the user ID, the third is information about the method of authentication, and the fourth is a set of scopes granted to the auth token.

```

var authToken = await UserAuthentication.signInUser(session, userInfo.id!,
    'myAuthMethod', scopes: {
        Scope('delete'),
        Scope('create'),
    },
);

```

```
});
```

The example above creates an auth token for a user with the unique identifier taken from the `userInfo`. The auth token preserves that it was created using the method `myAuthMethod` and has the scopes `delete` and `create`.

INFO

The unique identifier for the user should uniquely identify the user regardless of authentication method. The information allows authentication tokens associated with the same user to be grouped.

Send auth token to client

Once the auth token is created, it should be sent to the client. We recommend doing this using an `AuthenticationResponse`. This ensures compatibility with the client-side authentication module.

```
class MyAuthenticationEndpoint extends Endpoint {
  Future<AuthenticationResponse> login(
    Session session,
    String username,
    String password,
  ) async {
    // Authenticates a user with email and password.
    if (!authenticateUser(session, username, password)) {
      return AuthenticationResponse(success: false);
    }

    // Finds or creates a user in the database using the User methods.
    var userInfo = findOrCreateUser(session, username);

    // Creates an authentication key for the user.
    var authToken = await UserAuthentication.signInUser(
      session,
      userInfo.id!,
      'myAuth',
      scopes: {},
    );

    // Returns the authentication response.
    return AuthenticationResponse(
      success: true,
      keyId: authToken.id,
      key: authToken.key,
      userInfo: userInfo,
    );
  }
}
```

The example above shows how to create an `AuthenticationResponse` with the auth token and user information.

Revoking authentication keys

Serverpod provides built-in methods for managing user authentication across multiple devices. These methods handle several critical security and state management processes automatically, ensuring consistent and secure authentication state across your servers. When using the authentication management methods (`signOutUser` or `revokeAuthKey`), the following key actions are automatically handled:

- Closing all affected method streaming connections to maintain connection integrity.
- Synchronizing authentication state across all connected servers.
- Updating the session's authentication state with `session.updateAuthenticated(null)` if the affected user is currently authenticated.

Revoking specific keys

To revoke specific authentication keys, use the `revokeAuthKey` method:

```
await UserAuthentication.revokeAuthKey(  
  session,  
  authKeyId: 'auth-key-id-here',  
);
```

Fetching and revoking an authentication key using AuthenticationInfo

To revoke a specific authentication key for the current session, you can directly access the session's authentication information and call the `revokeAuthKey` method:

```
// Fetch the authentication information for the current session  
var authId = session.authenticated?.authId;  
  
// Revoke the authentication key if the session is authenticated and has an authId  
if (authId != null) {  
  await UserAuthentication.revokeAuthKey(  
    session,  
    authKeyId: authId,  
  );  
}
```

Fetching and revoking a specific authentication key for a user

To revoke a specific authentication key associated with a user, you can retrieve all authentication keys for that user and select the key you wish to revoke:

```
// Fetch all authentication keys for the user  
var authKeys = await AuthKey.db.find(  
  session,
```

```

    where: (t) => t.userId.equals(userId),
  );

  // Revoke a specific key (for example, the last one)
  if (authKeys.isNotEmpty) {
    var authKeyId = authKeys.last.id.toString(); // Convert the ID to string
    await UserAuthentication.revokeAuthKey(
      session,
      authKeyId: authKeyId,
    );
  }
}

```

Removing specific tokens (direct deletion)

```

await AuthKey.db.deleteWhere(
  session,
  where: (t) => t.userId.equals(userId) & t.method.equals('username'),
);

```

⚠ WARNING

Directly removing authentication tokens from the `AuthKey` table bypasses necessary processes such as closing method streaming connections and synchronizing servers state. It is strongly recommended to use `UserAuthentication.revokeAuthKey` to ensure a complete and consistent sign-out.

Signing out all devices

The `signOutUser` method signs a user out from all devices:

```

await UserAuthentication.signOutUser(
  session,
  userId: 123, // Optional: If omitted, the currently authenticated user will be signed
  out
);

```

This method deletes all authentication keys associated with the user.

Signing out a specific user

In this example, a specific `userId` is provided to sign out that user from all their devices:

```

// Sign out the user with ID 123 from all devices
await UserAuthentication.signOutUser(
  session,
  userId: 123,
);

```

```
);
```

Signing out the currently authenticated user

If no `userId` is provided, `signOutUser` will automatically sign out the user who is currently authenticated in the session:

```
// Sign out the currently authenticated user
await UserAuthentication.signOutUser(
  session, // No userId provided, signs out the current user
);
```

Creating a logout endpoint

To sign out a user on all devices using an endpoint, the `signOutUser` method in the `UserAuthentication` class can be used:

```
class AuthenticatedEndpoint extends Endpoint {
  @override
  bool get requireLogin => true;

  Future<void> logout(Session session) async {
    await UserAuthentication.signOutUser(session);
  }
}
```

Client setup

The client must store and include the auth token in communication with the server. Luckily, the client-side authentication module handles this for you through the `SessionManager`.

The session manager is responsible for storing the auth token and user information. It is initialized on client startup and will restore any existing user session from local storage.

After a successful authentication where an authentication response is returned from the server, the user should be registered in the session manager through the `sessionManager.registerSignedInUser(...)` method. The session manager singleton is accessible by calling `SessionManager.instance`.

```
var serverResponse = await caller.myAuthentication.login(username, password);

if (serverResponse.success) {
  // Store the user info in the session manager.
  SessionManager sessionManager = await SessionManager.instance;
  await sessionManager.registerSignedInUser(
    serverResponse.userInfo!,
```

```
        serverResponse.keyId!,  
        serverResponse.key!,  
    );  
}
```

The example above shows how to register a signed-in user in the session manager.